

Facilitator Training Manual

Every module, plus both full tutorials, in one document you can read without a screen

This is the paper twin of your training course and both student tutorials. Read it cover to cover once before Friday, ideally with a Codespace open so you can actually try the hands-on bits as you go. Everything here is the same content as the interactive versions — nothing's been simplified away.

Contents

- Part 1 — The Fundamentals (GitHub, Codespaces, Copilot, Nova)
- Part 2 — Nova vs Plain Copilot, condensed
- Part 3 — Full Bug Tutorial (do this yourself)
- Part 4 — Full Build Tutorial (do this yourself)
- Part 5 — Narrating Live & Common Questions

1 — What GitHub actually is

A repository ("repo") is a folder of files where every saved change is kept forever as a numbered commit. You can always go back to any earlier version — nothing is truly lost by editing it.

Your workshop repo contains `bank-dashboard.html`, `starter-template.html`, `challenge.js`, and two configuration files. That's it — a folder with a memory.

Ticking "Template repository" in Settings adds a **Use this template** button, letting anyone make their own full copy under their own account instantly. That's how every student gets an identical, independent copy without ever touching yours.

TRY IT

Open your repo, click "commits" near the top of the file list — every upload you've made is there, in order.

Q: If a student breaks their copy completely, is your original affected?

No. Each copy is completely independent.

2 — What a Codespace actually is

Not your computer — a borrowed one. Clicking "Create codespace" starts a real, temporary Linux machine in a data centre, installs VS Code and your `devcontainer.json` extensions, and streams the editor into the browser. The browser is just a window onto it.

WHY THIS MATTERS

If a Codespace misbehaves, delete it and make a new one — you're not "reinstalling," just asking for a fresh borrowed computer.

TRY IT

Open a Codespace, open a terminal, type `whoami` then `hostname` — confirms it's a real, separate machine.

3 — What Copilot actually does — three modes

Inline completions (Tab-complete) — type a comment, Copilot suggests code as ghost text, press Tab to accept. Prediction, not conversation.

Ask/Chat mode — a proper conversation. Doesn't touch files directly. This is where Nova's Bug Hunt restraint works best — it's built for conversation, not action.

Agent mode — reads and edits files directly, autonomously, multi-step. Nova's instructions still load here, but because Agent mode's whole purpose is to *act*, it's more likely to just fix things when asked. Nova has an explicit hard rule for this: one real fix, then stop and explain, during the Bug Hunt specifically.

THE ONE THING WORTH KNOWING COLD

Copilot is confidently wrong a meaningful amount of the time. Always test what it gives you.

WORTH TESTING BEFORE FRIDAY

Switch to Agent mode yourself, ask it to "fix the bugs in this file" during a mock Bug Hunt. See if it holds back as intended.

4 — What Nova adds, and why

Nova is not a separate tool — it's `.github/copilot-instructions.md`, a text file Copilot Chat reads automatically in your repo. It changes behaviour without changing which product students use.

See Part 2 below for the full, evidence-based comparison against plain Copilot.

Same tool, one extra file. Plain Copilot answers from general training with no idea it's talking to a 13-year-old mid-exercise. Nova reads the same repo's instructions file and adjusts.

EVIDENCE — BUG HUNT: "ARE THERE ANY BUGS IN THIS CODE?"

PLAIN COPILOT

Lists several or all bugs directly — no reason not to.

NOVA

"Yes, several! Look at the page first — does anything look wrong? Then use it — click things, type things. What breaks?"

EVIDENCE — OFF-TOPIC REQUEST

PLAIN COPILOT

Just tries to help with whatever's asked.

NOVA

"Let's get back to your feature — what part are you working on right now?"

HONEST LIMITS

It's an instruction, not a lock — a persistent student could still get more out of it than intended. It doesn't learn during the day — every chat starts fresh from the same file. It doesn't make Copilot more technically correct, only more appropriately toned.

THE HONEST VERDICT

Nova is genuinely better for this room — not more powerful, more *appropriate*. It protects the Bug Hunt, gets out of the way during Build, talks to students like students. Not a guarantee against misuse — for a supervised one-day workshop, that's the right bar, and it clears it.

Do this in a real Codespace, not just on paper. This is the exact tutorial students follow. Actually run it before Friday.

The method — four steps, every time

1. **Look** — open the preview. Does anything look visually wrong?
2. **Use** — click every button, type in every field. Does something not respond?
3. **Read** — open the code, read it slowly, line by line.
4. **Test** — make your fix, save, check the preview again. Don't assume — look.

Worked example (not a real dashboard bug)

A button is supposed to show a greeting. Click it — nothing happens, no error.

1 Look → nothing obviously wrong

The button looks normal. Not a visual bug — move on.

2 Use → confirmed broken

Click it — nothing happens. Now you know where to look.

3 Read → find it

```
<button onclick="sayHi()">Click me</button>

function sayHu() { ← should be sayHi
  alert("Hi!");
}
```

One letter different. JavaScript can't find a function that doesn't exist, so it silently does nothing.

4 Fix + test

Click before the `u`, delete it, type `i`. Save (`Ctrl+S`). Switch to Live Server, click the button — a popup says "Hi!"

Now for real — the actual dashboard

Same four steps, no answers given here on purpose — this is exactly what students see.

LOOK

Scroll the real preview top to bottom slowly. Anything missing, invisible, or misplaced?

USE

Click every button, type in every field. Then deliberately: leave a field empty, type letters where numbers go, type a negative number, type a huge number.

READ

Open the file, read HTML then CSS then JS, one line at a time. Press F12, check the Console tab for red errors while using the page.

FIX + TEST

Click the exact spot, correct it, save, switch to Live Server, redo the same action that was broken. Log it in the Bug Test Plan either way.

THE 10 REAL ANSWERS (FACILITATOR ONLY — NEVER SHOWN TO STUDENTS)

#	BUG	FIX
1	Missing closing tag	Add <code></div></code> after balance footer
2	Invisible balance	<code>color: #0F1340</code> → <code>FFFFFF</code>
3	Wrong transaction sign	Coffee: <code>credit</code> → <code>debit</code>
4	Wrong input type	<code>type="text"</code> → <code>"number"</code>
5	Button typo	<code>onlick</code> → <code>onclick</code>
6	Wrong operator	<code>balance + amount</code> → <code>- amount</code>
7	Element ID typo	<code>"balance-display"</code> → <code>"balance-displaiy"</code>
8	NaN on empty input	Add <code>isNaN()</code> check
9	Wrong interest formula	<code>* rate</code> → <code>* (rate/100)</code>
10	No overdraft check	Add <code>if (amount > balance)</code> guard

CATEGORIES WORTH KNOWING

Typos in names (read character by character) · colours that hide things · maths that's backwards · missing checks against silly input.

GETTING UNSTUCK

"I clicked this button and nothing happened — what could cause that?" · "The console shows this error: [paste it]. What does it mean?" · "I expected X, but Z happened instead. Why?"

Build the Savings Tracker for real, from a blank file. Same 10 steps students follow.

Setup

New File → name it `practice-feature.html` — genuinely empty. Open with Live Server too.

1 Basic page skeleton

```
TYPE
<!-- a basic HTML5 page with a title
"NovaPay — Savings Goal Tracker" -->
```

Tab to accept, save. Blank page, correct tab title — nothing visible yet, that's right.

2 Colour system

```
TYPE (INSIDE <HEAD>, A <STYLE> BLOCK)
/* a NovaPay colour palette using CSS variables:
purple accent, green success colour, light grey
background, dark text, plus a card style with
rounded corners and a soft shadow */
```

Save. Still nothing visible — the variables exist, nothing uses them yet.

3 NovaPay header bar

```
TYPE (INSIDE <BODY>)
<!-- a sticky header bar with the NovaPay logo
on the left, using the accent colour -->
```

Save, check Live Server — a header bar appears, styled from Step 2.

4 Input card

```
TYPE (BELOW THE HEADER)
<!-- a card with three inputs: goal name (text),
target amount (number), amount saved so far
(number), and a button labelled "Update my goal" -->
```

Save — a styled card with 3 inputs and a button appears. Clicking does nothing yet.

FOUNDATION CHECK

Styled page, header, input form — built from nothing. Everything from here reuses this.

5 Progress bar CSS

```
TYPE (IN <STYLE>)  
/* progress bar container and fill styles,  
using the colour variables from before */
```

Save. Nothing visible yet — no HTML uses it.

6 Result message CSS

```
TYPE  
/* styles for a result card showing  
a large amount remaining and a status message */
```

Save.

7 Progress bar HTML

```
TYPE (BELOW INPUT CARD)  
<!-- savings goal progress card with a  
labelled progress bar and percentage -->
```

Save, check Live Server — a progress bar card appears (0%).

8 Result card HTML

```
TYPE (BELOW THAT)  
<!-- result card showing amount left to save  
and a congratulations message if goal met -->
```

Save — a second card appears.

MILESTONE CHECK

Header, inputs, button, progress bar, result area — all visible, none working yet. On track.

9 The logic

Add a `<script>` tag before `</body>`.

TYPE

```
// function calculateProgress(savedAmount,  
//   targetAmount) that returns the percentage  
//   saved, capped at 100, rounded to 1 decimal
```

THEN TYPE

```
// function updateDisplay(goalName, targetAmount,  
//   savedAmount, percentage) that updates the  
//   progress bar width, percentage text,  
//   amount remaining, and shows a  
//   congratulations message if goal is met
```

Save both. Still nothing visible — neither function is called yet.

10 Wire it together — the big moment

```
function updateGoal() {  
  // YOUR CODE GOES HERE  
}
```

REPLACE THE COMMENT WITH

```
// read goalName, targetAmount and savedAmount  
// from the inputs, validate them, calculate  
// progress, and update the display
```

Save. Switch to Live Server, fill in real numbers (e.g. "New trainers", 80, 50), click the button.

SHOULD SEE

Progress bar animates to ~62%, result card shows "£30.00 left to save."

IF NOTHING HAPPENS

F12 → Console → look for red text → ask Nova with the exact error.

Testing & starting your own idea

Try: empty fields, a savings amount bigger than the target, a negative number. Log results in the Feature Test Plan.

Starting your own feature: New File → name it for your idea → copy the `<style>` section and header from `practice-feature.html` (the file you just built) → build with the same method. Reusing your own earlier work is normal developer practice, not cheating.

Narrating this live

You don't need new material — you need confidence. The full script is in the Master Script; what this manual gives you is the ability to say those lines from experience, not cold. You can honestly say "I did this exact tutorial myself" — because you did.

THE ONE HABIT WORTH BUILDING

When you genuinely don't know an answer, say so and open your own Copilot Chat live, in front of the student, and work it out together.

Common questions you'll get asked

"Is Copilot going to take my job one day?"

It changes what the job looks like — someone still has to know what to ask for and whether the answer's right. That's the skill being taught today.

"Why can't Nova just fix all the bugs for me?"

Deliberate — it hints rather than solves during Bug Hunt. Finding the bug is the point, not just having it fixed.

"Is my code actually running somewhere, or is it fake?"

Completely real — see Part 1, Module 2. A genuine, temporary computer.

"What happens to my Codespace after today?"

It sits idle and gets cleaned up eventually, but the repo — the actual files — stays on their GitHub account permanently.

Same content as [facilitator-training.html](#), [how-to-find-fix-bugs.html](#), and [how-to-build-your-feature.html](#) — this is just the version you can read without a screen.